

- *local*: The query returns data from the instance with no guarantee that the data has been written to a majority of the replica set members.
- *available*: This is identical to the 'local' in unsharded collections. But in a sharded cluster we have orphaned documents. These documents also exist in chunks on other shards because of failed migrations or incomplete migration cleanup due to abnormal shutdown. In 'local', the reads require communication with primary shard (if read is on secondary) or the config servers to service the read. Whereas the 'available' option does not contact the shard's primary or the config servers for updated metadata and may return orphan documents.
- *majority*: This query only returns data that was written to most voting nodes and will not be rolled back.
- *linearizable*: The query returns data that reflects all successful majority acknowledged writes that completed prior to the start of the read operation. It may wait for concurrently executing writes to propagate to a majority of replica set members before returning results.
- *snapshot*: Each write is synchronised across the host based on the time. The snapshot option reads from the latest synchronised time from all the nodes.
- **Write concern**: Write concern describes the level of acknowledgment requested from MongoDB for write operations to a standalone *mongod* or to replica sets or to sharded clusters. The levels are:
 - *majority*: Requests acknowledgement that write operations have propagated to the primary based on the calculated majority number for the secondary voting members. As an example, if we have a P-S-S replica set, the write should be propagated to primary and one secondary. Similarly, if we have a P-S-A replica set, the write should have been propagated to primary and secondary as well.
 - *<number>*: Requests acknowledgment that the write operation has propagated to the specified number of *mongod* instances. No acknowledgment for the write operation with 'w' is set to 0.
 - *<custom write concern name>*: Requests acknowledgment that the write operations have propagated to tagged members that satisfy the custom write concern defined in *settings.getLastErrorModes*.

Sharding

This is a method for distributing data across multiple machines. As all the data cannot be saved in a single disk, we need to have horizontal scaling. Sharding is the process of enabling horizontal scaling seamlessly beyond the hardware limits of the single server. Shards are used to store data in distributed systems using the shard key. The shard key determines how

data is distributed across a sharded cluster. As we can modify the shard key, MongoDB will automatically rebalance data across shards as needed without manual intervention. As seen in Figure 1, it's advisable to have a separate replica set for each shard, providing high availability and data consistency.

Chunks are subsets of shared data. MongoDB separates sharded data into chunks that are distributed across the shards in the shared cluster. Each chunk has an inclusive lower and exclusive upper range based on the shard key. A balancer specific for each cluster handles the chunk distribution. Sharding options are:

Ranged sharding: Documents are partitioned across shards according to the shard key value.

Hashed sharding: Documents are distributed according to an MD5 hash of the shard key value, providing even distribution.

Zoned sharding: Allows developers to define specific rules governing data placement in a sharded cluster.

 **Note**: Sharding requires careful planning, since the shard key directly impacts the overall performance of the underlying cluster, as it is used to identify all the documents within the collections.

Config server

Config servers store the cluster's metadata. This data contains a mapping of the cluster's dataset to the shards. The metadata includes the list of chunks on every shard and the ranges that define the chunks. The query router (*mongos* process) uses and caches this metadata to redirect target operations to specific shards. Config servers need to be placed in the replica set to enable consistency and resiliency, which can be up to 50 instances.

To be deployed as a replica set, the config servers must run the WiredTiger storage engine. If the config server replica set loses its primary and cannot elect it, the cluster's metadata becomes read only. You can still read and write data from the shards, but no chunk migration or chunk splits will occur until the replica set can elect a primary. In a sharded cluster, *mongod* and *mongos* instances monitor the replica sets in the sharded cluster (e.g., shard replica set, config server replica set).

Daemon *mongod*

mongod is the primary daemon process in Linux for the MongoDB system. It handles data requests, manages data access, and performs background management operations. The data is usually stored in */data/db*. Also, we cannot have two mongoDB instances running on the same port 27017.

Daemon *mongos*

The *mongos* instances provide the interface between the client applications and the sharded cluster. From the config server the